Enabling Efficient Intermittent Computing on Brand New Microcontrollers via Tracking Programmable Voltage Thresholds

Khakim Akhunov
University of Trento
Trento, Italy
khakim.akhunov@unitn.it

Eren Yildiz
Ege University
Izmir, Turkiye
eren.yildiz@ege.edu.tr

Kasim Sinan Yildirim
University of Trento
Trento, Italy
kasimsinan.yildirim@unitn.it

ABSTRACT

Modern off-the-shelf low-power microcontrollers (MCUs) are optimized to meet the computational requirements of data- and compute-intensive embedded artificial intelligence applications. However, they are not intended for batteryless operation; therefore, they lack fast and low-power non-volatile memory in their architecture. This memory is essential for backup and recovery operations during intermittent execution due to frequent power failures. Connecting an external non-volatile memory to these MCUs exposes a significant time and energy overhead, making them inefficient and even useless for batteryless applications. In this paper, we answer how to enable the adaptation of the brand-new off-the-shelf low-power AI MCUs to the intermittent computing paradigm. To this end, we present a new configurable low-power circuit that brings energy awareness, which is exploited by a novel backup policy that reduces the number of backups significantly. Our evaluation shows that the proposed backup technique reduces the execution latency by 40%, eliminating unnecessary backups and hence decreasing the intermittent computing systems' throughput significantly.

CCS CONCEPTS

• **Hardware** → *PCB design and layout*; • **Computer systems organization** → **Embedded software**.

KEYWORDS

Batteryless Platforms, Intermittent Computing, Voltage Monitoring

ACM Reference Format:
Khakim Ahunov, Eren Yildiz, Kasim Sinan Yildirim. 2023. Enabling Efficient Intermittent Computing on Brand New Microcontrollers via Tracking Programmable Voltage Thresholds. In *11th International Workshop on Energy Harvesting & Energy-Neutral Sensing Systems (ENSsys' 23), November 12, 2023, Istanbul, Turkiye.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3628353.3628547

1 INTRODUCTION

Batteryless devices rely solely on energy harvested from ambient sources and promise to rid sensing, computing, and communicating nodes of bulky, nondurable, and contaminating batteries [35]. The energy harvesters of such devices can convert electromagnetic, kinetic, thermal, or luminous power into electrical energy and collect it in tiny energy buffers, e.g., capacitors. Generally, the capacity of such small buffers is enough to perform only a limited chunk of a whole computational task. Moreover, frequent depletion of the capacitor leads to frequent power failures, causing *intermittent execution*. The intermittent nature of execution causes the loss of intermediate results and architectural state unless explicitly backed up in non-volatile memory before a power failure and restored after power is revived. Most existing work on intermittent computing focuses on software and hardware support for efficient backup and recovery operations [18].

Today's de facto intermittent computing platforms are built around TI MSP430FR series microcontrollers [37], which have *embedded* and *fast* non-volatile memory, i.e., FRAM [22] (ferroelectric RAM), in their architecture. Compared to FLASH memory, FRAM is low-power and significantly faster, with an almost unlimited number of read and write cycles [15, 21]. Therefore, FRAM combines the benefits of non-volatile memories with the higher performance of SRAM. This design is crucial for energy-efficient backup and recovery operations during intermittent execution since it reduces the energy overhead during data transfers between the non-volatile memory and processor. However, most off-the-shelf low-power MCUs are not intended to be used for intermittent computing; therefore, they do not have fast and embedded non-volatile memories in their architecture.

As an example, MAX32666 [23] from Maxim Integrated features a dual low-energy Arm Cortex-M4F processor, providing computational power and parallelism support for embedded AI applications. It consumes approximately 3mA at 96MHz and 10µA in deep sleep mode, making it a perfect fit for batteryless applications as well. The only way to exploit this MCU for batteryless applications is to introduce an external non-volatile memory, such as FRAM [34] connected via a serial peripheral interface (SPI). Nevertheless, using external non-volatile memory is inefficient due to the significant energy overhead of serial communication, making backup and recovery operations energy-expensive [3].

Problem Statement. The low-power MCU market is growing with the boosting effect of TinyML [33, 38], introducing several brand-new MCUs [2, 10, 14, 16] to support low-power intelligence on the edge. However, these MCUs are designed for continuous operation; therefore, they do not include a fast non-volatile memory embedded in their architecture. Connecting an external FRAM to these MCUs and employing existing software support for intermittent computing [1, 7, 8, 13, 39–41] is inefficient, making them not preferable for batteryless applications. Without systems support, intermittent computing will be doomed to use obsolete MSP430FR series



This work is licensed under a Creative Commons Attribution International 4.0 License.
ENSsys' 23, November 12, 2023, Istanbul, Turkiye
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0438-3/23/11.
https://doi.org/10.1145/3628353.3628547

MCUs, which have limited computational power and capabilities, preventing the widespread adoption of brand-new MCUs in the development of batteryless intelligence on the edge [17].

Contributions. This paper aims to close the gap between batteryless energy-harvesting applications that execute intermittently because of frequent power failures and low-power modern MCUs designed for continuous operation. To this end, we provide the necessary system support for the efficient adoption of low-power MCUs for intermittent computing via the following contributions:

(1) Energy-Tracker. We introduce a *configurable* low-power circuit, named **TETRA** (**Thre**E **T**heshold t**RA**cker), to monitor and keep track of the energy stored in the capacitor. TETRA generates three signals to interrupt execution: (1) when the energy in the capacitor reaches the *operating threshold* of the system; (2) when the energy reaches a desired *charging threshold*; and (3) when the energy is equal to the given *checkpoint threshold*.

(2) Energy-Tracker Triggered Backups. We introduce TETRA-driven backups, a novel technique for backup and recovery that reduces the frequency and, in turn, the number of backups significantly. Existing software support for intermittent computing backs up the computational state pessimistically (e.g., at each task completion [7]). Contrarily, TETRA-driven backups are performed when it is unavoidable, i.e., there is not enough ambient power to charge the capacitor. When the energy stored in the capacitor drops below the charging threshold, TETRA triggers an interrupt, which puts the MCU in sleep mode, retaining volatile memory content. While in sleep mode, the MCU waits for the capacitor to be charged without performing a checkpoint. *The backup operation is performed if and only if a power failure is unavoidable*, i.e., the ambient power is smaller than the sleep mode power consumption, i.e., the stored energy in the capacitor drops below the backup threshold and TETRA triggers the interrupt.

With this system support, MCUs use only SRAM during computation. External FRAM is accessed exclusively for backup and recovery operations. The system operates *continuously* without performing any backup as long as the ambient energy is above the sleep mode power consumption of the platform, exploiting the energy efficiency of their de facto volatile architecture.

2 BACKGROUND AND MOTIVATION

Batteryless platforms use energy harvesting to charge a small capacitor. The harvester's output power is not constant due to sporadic ambient power. If the output power is lower than the power consumed, the voltage level in the capacitor decreases gradually and drops below the operating threshold, causing a power failure. When a power failure occurs, batteryless computing platforms lose the volatile computational state stored in their registers, stack, and global variables in memory. Frequent power failures lead to intermittent operation, making non-volatile memory an essential component to back up and recover computational state.

As of now, almost all intermittent computing platforms, e.g., Flicker [19] and Camaroptera [9], include TI MSP430FR series MCUs [37]. These MCUs include a main bus that interconnects a *single-core* processor, main memory (SRAM), FRAM, and other peripherals. The processor can access FRAM in an energy-efficient manner by using the optimized interconnecting bus since FRAM is an embedded component of the internal architecture of the MCU.

2.1 Intermittent Computing Backup Solutions

Intermittent programs access FRAM frequently to manipulate persistent variables. A power failure can leave persistent variables partially updated and anti-dependencies might cause non-volatile memory inconsistencies [20, 31, 32]. Intermittent computing solutions propose backup strategies to maintain computational progress and prevent non-volatile memory inconsistencies [5, 30].

The backup strategies can be classified into two spheres: checkpoints and task-based programming. In checkpoint-based approach [6, 25, 26, 32, 41], a snapshot of the volatile computation state, i.e., the values of registers, stack, and global variables, is logged in FRAM. When the device reboots after its capacitor charge reaches an operating threshold, it recovers its computational state by using the last successful checkpoint and progresses the interrupted computation. The frequency of checkpoints depends on the checkpoint policy. For instance, TICS [25] can employ a fixed period (i.e., timer-driven) checkpointing approach, in which at each user-selected interval (e.g., 100 ms) a checkpoint is taken. Differently, RockClimb [6] employs a compiler-driven approach. The program is divided into regions based on the capacitor size. The capacitor's charge is measured before executing a region. The region is only executed if the stored energy is enough to execute that region. At the end of each region, a lightweight checkpoint might be taken (e.g., only some registers), depending on the register-level dependency between successive regions.

Alternatively, task-based programming [28, 39] requires applications to be manually partitioned into a set of atomic tasks. The programmer identifies the control flow and the data flow among the tasks. For instance, Chain [7] maintains inputs and outputs of tasks separately in FRAM, removing anti-dependencies and making tasks idempotent. Differently, InK [39] logs each memory write to roll back FRAM modifications upon power failures. At each task transition, modifications of the tasks are committed to FRAM, which can be conceptually seen as a checkpoint.

2.2 Challenges of Using Brand New MCUs

Considering the computational power, parallelism support, and power efficiency of MAX32666, it can be a suitable candidate for modern battery-free AI applications. However, integrating new MCUs into intermittent computing poses challenges.

Lack of embedded non-volatile memory. Many low-power MCUs are not designed for intermittent computing; they are designed for continuous operation. Therefore, their main memory is volatile (e.g., SRAM) and they do not have fast non-volatile memory. The devices have FLASH memory to store program code. However, FLASH memories are not suitable for intermittent computing due to their high energy requirements, low speed, and limited write endurance. The only way to exploit contemporary low-power platforms for intermittent computing is to introduce an external non-volatile memory, such as an external SPI-based FRAM [34]. However, this configuration is inefficient due to its significant energy overhead, making backup and recovery operations expensive.

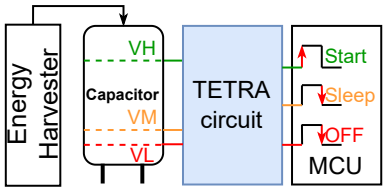


Figure 1: TETRA energy level monitoring strategy.

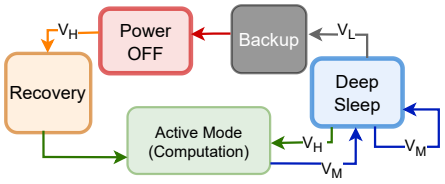


Figure 2: Finite state machine of TETRA computational flow.

Frequent non-volatile memory access. Current intermittent computing solutions require frequent backup and recovery operations, leading to frequent non-volatile memory accesses, as shown in [41, Table 5]. For instance, task-based systems (e.g., [39]) backup computational state pessimistically at each task transition. Similarly, checkpoint-based systems backup at either statically-defined program points (e.g., [41]) or when power failure is imminent (e.g., [4]). When these solutions are applied using external non-volatile memory, low-power MCUs spend a significant amount of time and energy on backup and recovery operations, decreasing system throughput significantly. Similarly, existing intermittent computing solutions frequently use non-volatile memory for maintaining and manipulating program data. For instance, task-based systems (e.g., [28, 39]) keep task-shared variables in FRAM. Several checkpointing systems (e.g., [6, 41]) also use FRAM as the main memory, making the program interact with non-volatile memory most of the time instead of SRAM. Hence, when these solutions are applied to brand-new MCU systems, frequent access to external non-volatile data brings significant overhead.

2.3 Our Differences from Prior Art

The study introduced by Lukosevicius et al. [27] is the closest to our work, as it presented the use of sleep mode for preventing redundant checkpoints by switching the MCU to sleep mode. However, the authors do not present a concrete circuit that supports this operation, in particular, their approach is not configurable. Moreover, the authors considered the de facto MSP430FR series MCUs to demonstrate the feasibility of their idea. Finally, the authors build their work on Hibernus [4], which uses FRAM as the main memory, leading to frequent FRAM access. Contrarily, our work introduced adjustable thresholds to switch to sleep mode, considering the power consumption of different configurations. Moreover, we use FRAM only for backup operations.

3 TETRA DESIGN AND IMPLEMENTATION

TETRA is designed to facilitate efficient intermittent computing through three crucial architectural decisions. First, we propose a lightweight voltage monitoring solution to adjust the system's power consumption based on ambient power availability. Second, we introduce energy awareness, enabled by our lightweight voltage monitoring, to delay power failures and postpone costly checkpoints as much as possible. Third, we allow non-volatile memory access only for backups, not during computation. Through these decisions, we successfully reduced the frequency of FRAM access. Thus, modern MCUs can harness their optimized and efficient volatile architecture during all operations, including memory-bound computations.

3.1 Energy-Aware Minimal Backups

Several existing studies, e.g., Samoyed [29] and QuickRecall [24], use a basic monitoring circuit to track the voltage level of the on-board energy storage capacitor. This approach allows the system to capture two predetermined thresholds: one for triggering backup and turning off the system (*VL*) and one for activating and restoring the system (*VH*). TETRA introduces a novel concept, including an additional voltage threshold (*VM*) between *VL* and *VH*. Figure 1 illustrates tracking the energy currently stored in the capacitor. When the voltage level of the capacitor reaches the VH level, the TETRA circuit generates the *Start* signal, which declares that $V_{in} \geq VH$. This action initiates a rising-edge interrupt within the MCU, meaning that the system has accumulated sufficient energy to wake up. Thus, the system restarts from the previous checkpoint and resumes the interrupted computation. When the voltage within the capacitor drops below VL, the TETRA circuit sends the *OFF* signal to backup and power down the system to recharge the capacitor until VH level. Furthermore, our system derives significant advantages from introducing the third voltage threshold (VM), which generates the *Sleep* signal. This threshold triggers an MCU falling-edge interrupt, serving as an early warning to the system to transition into the lowest power mode while preserving volatile memory content.

3.2 Restricted FRAM Access: SRAM as Main Memory for Intermittent Computing

TETRA uses the SRAM as the main memory and strictly prohibits external FRAM access by programs. This approach facilitates new MCUs to operate under stringent energy-efficient constraints, effectively eliminating FRAM access and substantially enhancing the efficiency of memory-bound operations. Checkpoints in TETRA are triggered as energy levels reach a predetermined critical threshold. This event allows the system to retain ongoing computations within volatile memory (SRAM) during normal operation. Saving temporary computation results and architectural state to non-volatile memory (FRAM) is reserved solely for moments when an imminent power failure is unavoidable, guaranteeing *uninterrupted system operation* during the execution and charging phases while avoiding energy-expensive checkpoints. This strategy improves system performance and reduces runtime power consumption. Moreover, it removes the need for complex compiler analysis or program transformation common in other intermittent computing techniques. Remarkably, despite employing a straightforward checkpointing strategy that backs up the entire SRAM and processor registers, TETRA attains significant performance advantages.

Figure 2 illustrates the state machine of the intermittent computing process following by TETRA. It is assumed that computation initiates exclusively with a fully charged capacitor. In TETRA's

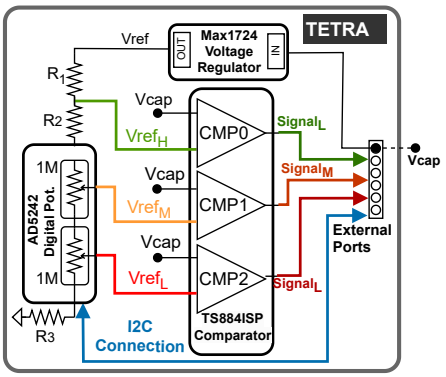


Figure 3: TETRA prototype comprises COTS components.

checkpointing strategy, SRAM serves as the primary memory, and access to FRAM is restricted until the V_L signal is triggered, preventing costly FRAM operations. To optimize energy usage and mitigate these costs, TETRA employs a V_M signal to protect SRAM data. When the V_M signal activates during the active mode, TETRA transitions the MCU into a deep sleep mode to postpone power failure. In cases where the average incoming ambient power exceeds the deep sleep mode's power consumption, the capacitor charges until it reaches the V_H voltage level, thus conserving energy associated with the resource-intensive backup operation, including costly FRAM operations. Conversely, when the average ambient power falls below the deep sleep mode threshold, TETRA resorts to a traditional checkpointing operation using the V_L signal to preserve SRAM data in FRAM. Following a power failure, the system restarts and recovers the latest saved checkpoint state once the capacitor voltage level reaches the V_H level again.

3.3 TETRA Hardware Prototype

We have developed the TETRA prototype using Commercial Off-The-Shelf (COTS) components, as illustrated in Figure 3. In our implementation of TETRA, we have integrated the Max1724 low-power DC-to-DC converter [12], which provides a regulated 3.3V output voltage to supply power to the elements of the TETRA block while also generating distinct reference voltages for various signal levels. To create V_{refL}, V_{refM}, and V_{refH}, we have designed a voltage divider circuit using resistors R1, R2 and R3 in conjunction with the AD5242-1MΩ dual programmable digital potentiometer [11], which references the output voltage of the Max1724. Rather than a fixed design, we used the digital potentiometer to be able to adjust VM and VL thresholds at runtime by the application, enabling flexibility for the dynamic adaptation of these thresholds. In our current implementation, we have selected 5MΩ, 750KΩ, and 3MΩ resistors for R1, R2, and R3, respectively, to set $V_{refL_{min}}$, $V_{refM_{min}}$, and V_{refH} at 1.25V, 1.35V, and 1.9V, respectively. Consequently, the TETRA software possesses the capability to configure both V_{refL} and V_{refM} within the ranges of 0.99V to 1.32V and 1.32V to 1.66V, respectively, by adjusting the divider ratios of the potentiometer via the I2C interface. The final component within the TETRA block is the TS884ISP quad nano power comparator [36]. This comparator is responsible for generating signals for the MCU when the capacitor voltage attains pre-defined reference voltages.

Table 1: List of the simulation parameters.

Parameter	Value	Parameter	Value
Operating frequency	96 MHz	Inter-core signal time	0.052 µs
Energy per clock cycle	0.113 nJ	Inter-core signal energy	0.576 nJ
Number of instructions	1 000 000	Voltage comparison time	0.0624 µs
Capacitor	1 mF	Voltage comparison energy	0.68 nJ
Active power	10 mW	Checkpoint time	96350 µs
Deep sleep power	0.033 mW	Checkpoint energy	829610 nJ
Boot time	500 µs	Recovery time	96350 µs
Boot energy	5450 nJ	Recovery energy	829610 nJ
FRAM read energy	829610 nJ	FRAM write energy	829610 nJ

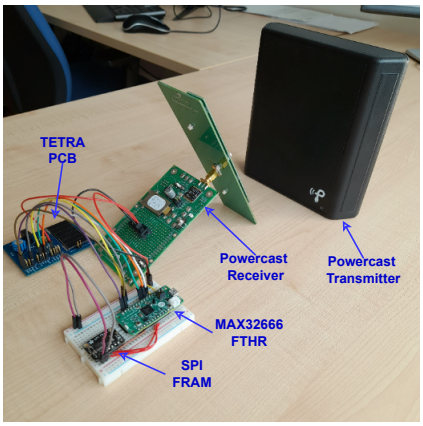


Figure 4: Evaluation setup.

4 EVALUATION AND RESULTS

To evaluate TETRA, we simulate its performance in an energy-harvesting environment and validate the system's achievements on a real hardware setup.

Simulation. We develop an in-house Python simulation to estimate performance and energy consumption, using parameters (see Table 1) obtained from the datasheets of the MAX32666 MCU [23], FRAM [34], and components mentioned in Section 3. The inter-core communication, checkpoint, and recovery overheads are estimated assuming that 32 bits are enough to deliver necessary information between cores and that the entire SRAM and CPU registers are backed up and recovered. The application comprises a set of randomly distributed instructions with different time and energy costs. The instructions are grouped into multiple non-parallelizable and parallelizable blocks randomly shuffled. To execute the application, we go through the instructions and drain the capacitor according to the instruction energy cost, optimistically assuming that at each clock cycle, one instruction is executed. We run the application under various ambient power levels, introducing diverse intensities of charging. We periodically check the energy level in the capacitor to take an action corresponding to a particular voltage threshold. The total execution time includes the instructions' execution time and other parameters presented in Table 1. We compare TETRA against state-of-the-art systems with just-in-time checkpoints (JIT) [1] and compiler-placed checkpoints (CP) [6] with SRAM as the main memory and FRAM as the memory only for backups.

Hardware Setup. The evaluation is based on the MAX32666 platform, as shown in Figure 4. The platform features 560KB SRAM

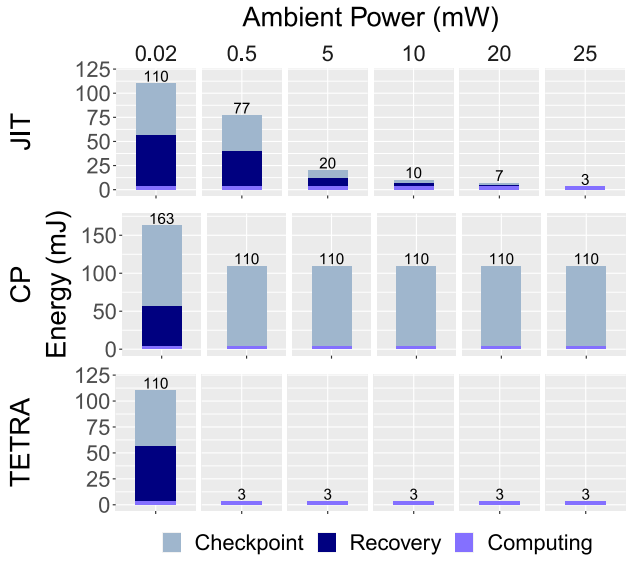


Figure 5: Energy evaluation.

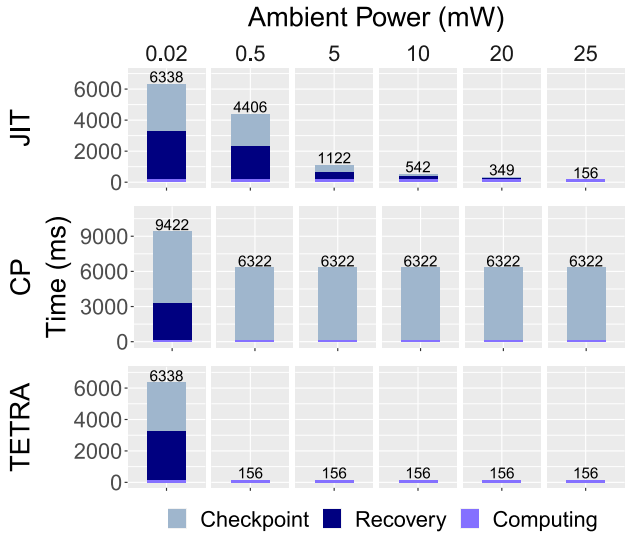


Figure 6: Execution time evaluation.

shared between two Cotrex-M4 cores running at 96MHz. We extend our evaluation setup with external 512KB SPI-based FRAM used only for backups for intermittent computing. As an energy-harvesting part, we use the Powercast TX91501-3W and P2110-EVB to collect RF energy in an internal 50mF capacitor and convey the harvested energy to MAX32666. The board is connected via I2C to the TETRA PCB to receive three signals for all the thresholds and to send back the signals adjusting VM and VL thresholds to a desired mode. The energy-harvesting kit has a booster converter that keeps the supplied voltage stable, 3.3V. However, the actual voltage outputting from the capacitor varies from 2V to 1.02V. We set 1.9V as the VH threshold, 1.35V, and 1.25V as the VM and VL thresholds. All the thresholds are determined based on the application and target platform. To use the full potential of the device, VH is set to its maximum operating voltage and VL is set to the value where the energy stored in the capacitor between VL and the minimum operating voltage of the device is enough to atomically handle the interrupt from TETRA and execute the worst case checkpoint. The basic rule for determining VM is having enough energy between VM and VL to handle the interrupt and switch the system to sleep mode. Note that setting VM too high takes energy away from operating mode (difference between VM and VH), which could be used for execution. To evaluate the performance of TETRA under different modes, we repetitively execute a convolution operation on a 32×32 single-channel image, applying a 2×2 kernel. Convolution is a widely used operation in digital signal processing and machine learning and involves executing multiply-and-accumulate (MAC) operations while sliding the kernel over the input data matrix. We count the number of MACs performed in 60 sec, calculate the number of MAC operations per second (MACOPS), and use this measure as a performance metric. Note that in our evaluation, we do not compare TETRA against FLASH- or internal FRAM-based intermittent systems since the superiority of FRAM over FLASH has been already shown in [15, 21], while the significant outperformance of internal FRAM over external one has been showcased in [5].

Table 2: Charging time comparison.

	Time (ms)					
	0.02 mW	*0.5 mW*	*5 mW*	*10 mW*	*20 mW*	*25 mW*
JIT	5489716	148932	2951	494	75	0
CP	5492698	227851	15784	4744	70	35
TETRA	5489716	7596	714	178	89	0

4.1 Simulation Results

In Figure 5, we show the breakdown of energy consumption of JIT, CP, and TETRA. We test environments with different constant levels of ambient power, from 0.02 to 25mW. As seen, when power is lower than the power consumption of sleep mode (i.e., 0.033), JIT and TETRA exhibit identical energy characteristics due to the same number of power failures, while CP consumes 48% more energy. Increasing ambient power leads to a linear decrease in JIT energy consumption. However, CP benefits only from eliminating recovery overhead because the checkpoints are predefined and performed before the system decides whether to power off or switch to power down mode and wait for charging (e.g., [6, see Section V]). This approach does not allow the system to avoid checkpoints and keeps the energy consumption at the same level even for higher ambient power. Conversely, TETRA spends energy only on computation, avoiding all the checkpoint and recovery overheads thanks to the tri-threshold approach, which never reaches the lowest voltage threshold with higher ambient power.

Figure 6 confirms that the JIT solution experiences fewer power failures when input power increases and the total checkpoint and recovery overhead becomes negligible compared to computation energy consumption. However, CP running an application on SRAM cannot fully benefit from increased ambient power since it needs to execute all the compiler-placed checkpoints. Being energy-aware, TETRA starts to benefit from the increased input power earlier compared to JIT. For example, even with 0.5mW, the execution time of our approach consists of only actual computation time, which is achievable by JIT only with 25mW of ambient power.

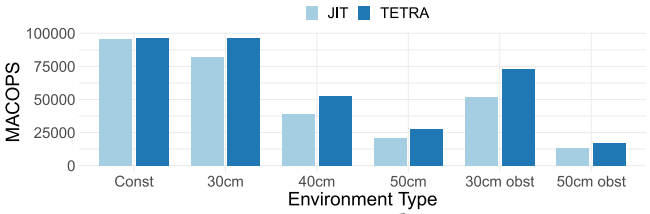
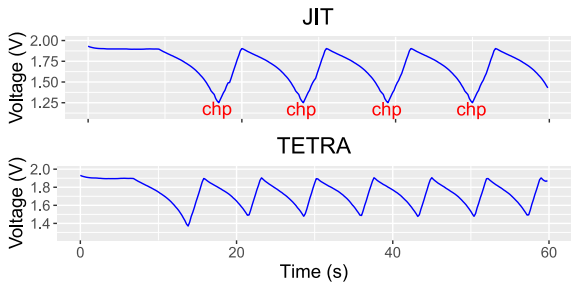
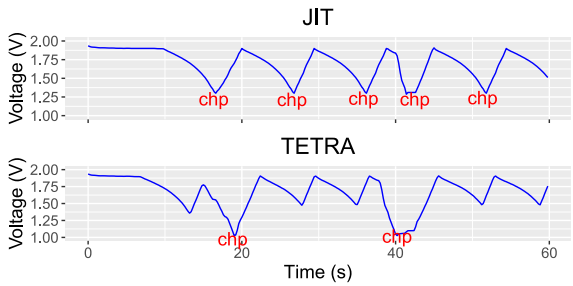


Figure 7: MACOPS evaluations.



(a) 30cm distance.



(b) 30cm distance with obstacle.

Figure 8: Dinamics of the capacitor voltage. *chp*–checkpoint.

While energy-efficient, robust, and able to run modern computational loads, energy-harvesting systems heavily depend on the strength of ambient power. One main component of the performance of an entire system is charging time. In Table 2, we compare the charging time for the evaluated approaches. The table shows that with the weakest power presented (i.e., 0.02mW), the charging time for the application with a 1mF capacitor can reach hundreds and thousands of seconds for all three approaches. However, with 0.5mW, charging time reduces to tens and hundreds of seconds for JIT and CP and to only several seconds for TETRA. Increasing input power further reduces the charging time, which reaches zero value with maximum power. Note that depending on the application and ambient power, the charging time can dramatically dominate in entire execution time. For example, TETRA under average 5mW spends 82% of application execution time for charging. However, TETRA still outperforms traditional JIT by 4.7× due to less time spent on checkpoints and recovery.

4.2 Experimental Evaluation

To validate the TETRA achievements in the simulation presented above, we run the convolution operation on real hardware (Figure 4) under six different conditions of input power: one condition with a constant power supply ensuring no power failures; three conditions of harvesting RF energy in different distances between the transmitter and receiver (30, 40, and 50 cm); additionally, two energy-harvesting scenarios with different distances and periodically appearing an obstacle between the transmitter and receiver (30cm obst and 50cm obst). To simulate an obstacle, we put a thin plastic plate (15×20cm) for 5 seconds between the RF transmitter and receiver every 15 seconds of application execution.

In Figure 7, we compare JIT and TETRA in MACOPS for the six scenarios mentioned above. Furthermore, during the execution of the MAC operations, we track the voltage of the capacitor, as shown in Figure 8 for 30cm and 30cm obst. As seen in Figure 7, both constantly powered solutions (i.e., with no power failures) perform identically. Positioning the RF transmitter 30cm away from the receiver reduces the amount of harvested energy, which causes power failures. However, compared to JIT, TETRA encounters no power failures (see Figure 8a), which increases the performance by 15%. The 40cm and 50cm distances significantly reduce MACOPS for both solutions due to increased charging time. However, increasing the distance also increases the number of power failures in conventional JIT, allowing TETRA to perform 1.3× faster. Violating the RF energy transmission by obstacles during the application execution causes power failures also in TETRA because input power drops below the power consumption of sleep mode. However, despite the obstacles, TETRA performs less often checkpoints compared to JIT (see Figures 8b), outperforming JIT by 40%.

5 CONCLUSION

The range of batteryless devices capable of tolerating frequent power failures is extremely limited due to the lack of a vital component of intermittent computing–built-in energy-efficient and fast non-volatile memory. This limitation forces designers of energy-harvesting systems to employ external non-volatile memory in combination with advanced modern MCUs, spending precious energy and time on external data movement and access. To tackle this problem, we proposed TETRA, an intermittent computing technique that uses energy awareness to avoid redundant accesses to non-volatile memory and exploits hard-won energy more efficiently. The technique is accompanied by a simple and compact circuitry that monitors and forwards the state of the capacitor to the MCU, which acts accordingly. Our simulation and real implementation results showed that TETRA can outperform state-of-the-art solutions by 1.4 times.

ACKNOWLEDGMENTS

Funded by the European Union | European Innovation Council

Funded by the European Union (project no. 101071179). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or EISMEA. Neither the European Union nor the granting authority can be held responsible for them.

REFERENCES

[1] Khakim Akhunov and Kasim Sinan Yildirim. Adamica: Adaptive multicore intermittent computing. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 6(3):1–30, 2022.

[2] ambiq. Apollo4 datasheet. Technical report, ambiq, 2023.

[3] Abu Bakar, Rishabh Goel, Jasper de Winkel, Jason Huang, Saad Ahmed, Bashima Islam, Przemysław Pawełczak, Kasım Sinan Yıldırım, and Josiah Hester. Protean: An energy-efficient and heterogeneous platform for adaptive and hardware-accelerated battery-free computing. In *Proceedings of the 20th ACM Conference on Embedded Networked Sensor Systems*, pages 207–221, 2022.

[4] Domenico Balsamo, Alex S Weddell, Anup Das, Alberto Rodriguez Arreola, Davide Brunelli, Bashir M Al-Hashimi, Geoff V Merrett, and Luca Benini. Hibernus++: a self-calibrating and adaptive system for transiently-powered embedded devices. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 35(12):1968–1980, 2016.

[5] Luca Caronti, Khakim Akhunov, Matteo Nardello, Kasım Sinan Yıldırım, and Davide Brunelli. Fine-grained hardware acceleration for efficient batteryless intermittent inference on the edge. *ACM Transactions on Embedded Computing Systems*, 2023.

[6] Jongouk Choi, Larry Kittinger, Qingrui Liu, and Changhee Jung. Compiler-directed high-performance intermittent computation with power failure immunity. In *2022 IEEE 28th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 40–54. IEEE, 2022.

[7] Alexei Colin and Brandon Lucia. Chain: Tasks and channels for reliable intermittent programs. In *Proc. OOPSLA*, pages 514–530, Amsterdam, Netherlands, 2016. ACM.

[8] Jasper De Winkel, Vito Kortbeek, Josiah Hester, and Przemysław Pawełczak. Battery-free game boy. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 4(3):1–34, 2020.

[9] Harsh Desai, Matteo Nardello, Davide Brunelli, and Brandon Lucia. Camaroptera: A long-range image sensor with local inference for remote sensing applications. *ACM Transactions on Embedded Computing Systems (TECS)*, 21(3):1–25, 2022.

[10] Analog Devices. Artificial intelligence microcontroller with ultra-low-power convolutional neural network accelerator, 2020.

[11] Analog Devices. Ad5242 - compatible 256-position digital potentiometers, 2022.

[12] Analog Devices. Adp5091 - step-up dc-dc converters in tsot and µdfn, 2022.

[13] Caglar Durmaz, Kasim Sinan Yildirim, and Geylani Kardas. Puremem: a structured programming model for transiently powered computers. In *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, pages 1544–1551, 2019.

[14] Eric Flamand, Davide Rossi, Francesco Conti, Igor Loi, Antonio Pullini, Florent Rotenberg, and Luca Benini. Gap-8: A risc-v soc for ai at the edge of the iot. In *2018 IEEE 29th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, pages 1–4. IEEE, 2018.

[15] Fujitsu. Fram. Technical report, Fujitsu, 2012.

[16] Angelo Garofalo, Manuele Rusci, Francesco Conti, Davide Rossi, and Luca Benini. Pulp-nn: accelerating quantized neural networks on parallel ultra-low-power risc-v processors. *Philosophical Transactions of the Royal Society A*, 378(2164):20190155, 2020.

[17] Graham Gobieski, Brandon Lucia, and Nathan Beckmann. Intelligence beyond the edge: Inference on intermittent embedded systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 199–213, 2019.

[18] Arda Goknil and Kasim Sinan Yildirim. Toward sustainable iot applications: Unique challenges for programming the batteryless edge. *IEEE Software*, 39(5):92–100, 2022.

[19] Josiah Hester and Jacob Sorber. Flicker: Rapid prototyping for the batteryless internet-of-things. In *Proceedings of the 15th ACM Conference on Embedded Network Sensor Systems*, pages 1–13, 2017.

[20] Matthew Hicks. Clank: Architectural support for intermittent computation. *ACM SIGARCH Computer Architecture News*, 45(2):228–240, 2017.

[21] Texas Instruments. Non-volatile memory: Flash & fram. Technical report, Texas Instruments, 2021.

[22] Texas Instruments. Fram - new generation of non-volatile memory, 2022.

[23] Maxim Integrated. Max32666 low-power arm cortex-m4 with fpu-based microcontroller with bluetooth 5 for wearables - maxim integrated, 2022.

[24] Hrishikesh Jayakumar, Arnab Raha, Woo Suk Lee, and Vijay Raghunathan. Quick-recall: A hw/sw approach for computing across power cycles in transiently powered computers. *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, 12(1):1–19, 2015.

[25] Vito Kortbeek, Kasim Sinan Yildirim, Abu Bakar, Jacob Sorber, Josiah Hester, and Przemysław Pawełczak. Time-sensitive intermittent computing meets legacy software. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 85–99, 2020.

[26] Brandon Lucia and Benjamin Ransford. A simpler, safer programming and execution model for intermittent systems. *ACM SIGPLAN Notices*, 50(6):575–585, 2015.

[27] Giedrius Lukosevicius, Alberto Rodriguez Arreola, and Alex S Weddell. Using sleep states to maximize the active time of transient computing systems. In *Proceedings of the Fifth ACM International Workshop on Energy Harvesting and Energy-Neutral Sensing Systems*, pages 31–36, 2017.

[28] Kiwan Maeng, Alexei Colin, and Brandon Lucia. Alpaca: Intermittent execution without checkpoints. *arXiv preprint arXiv:1909.06951*, 2019.

[29] Kiwan Maeng and Brandon Lucia. Supporting peripherals in intermittent systems with just-in-time checkpoints. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 1101–1116, 2019.

[30] Sebin Shaji Philip, Roberto Passerone, Kasim Sinan Yildirim, and Davide Brunelli. Intermittent computing emulation of ultralow-power processors: Evaluation of backup strategies for risc-v. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(1):82–94, 2022.

[31] Benjamin Ransford and Brandon Lucia. Nonvolatile memory is a broken time machine. In *Proceedings of the workshop on Memory Systems Performance and Correctness*, pages 1–3, 2014.

[32] Benjamin Ransford, Jacob Sorber, and Kevin Fu. Mementos: System support for long-running computation on RFID-scale devices. In *Proc. ASPLOS*, Newport Beach, CA, USA, 2011. ACM.

[33] Ramon Sanchez-Iborra and Antonio F Skarmeta. Tinyml-enabled frugal smart objects: Challenges and opportunities. *IEEE Circuits and Systems Magazine*, 20(3):4–18, 2020.

[34] Fujitsu Semiconductor. Mb85rs64 fram, 2023.

[35] Esther Shein. A battery-free internet of things. *Communications of the ACM*, 64(7):16–18, 2021.

[36] STMicroelectronics. Ts884 - quad nanopower comparators, 2017.

[37] Texas Instruments. Msp430fr58xx, msp430fr59xx, msp430fr68xx, and msp430fr69xx family user's guide. http://www.ti.com/lit/ug/slau367o/slau367o.pdf, 2019. Last accessed: September 2023.

[38] Pete Warden and Daniel Situnayake. *Tinyml: Machine learning with tensorflow lite on arduino and ultra-low-power microcontrollers*. O'Reilly Media, 2019.

[39] Kasım Sinan Yıldırım, Amjad Yousef Majid, Dimitris Patoukas, Koen Schaper, Przemyslaw Pawelczak, and Josiah Hester. Ink: Reactive kernel for tiny batteryless sensors. In *Proceedings of the 16th ACM Conference on Embedded Networked Sensor Systems*, pages 41–53, 2018.

[40] Eren Yildiz, Saad Ahmed, Bashima Islam, Josiah Hester, and Kasim Sinan Yildirim. Efficient and safe i/o operations for intermittent systems. In *Proceedings of the Eighteenth European Conference on Computer Systems*, pages 63–78, 2023.

[41] Eren Yıldız, Lijun Chen, and Kasim Sinan Yıldırım. Immortal threads: Multithreaded event-driven intermittent computing on {Ultra-Low-Power} microcontrollers. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 339–355, 2022.